Kubernetes Fundamentals Course Summary

- Introduction and Course Overview 🎯
  - Instructor Chad is CKA and CKAD certified with 20 years of education experience.
  - The course covers foundations needed for CKA/CKAD exams or general Kubernetes proficiency.
  - Hands-on labs, AI support, and full courses available at Alta3 Academy.
  - Timestamps available for advanced users; beginners advised to follow the presented order.
- Kubernetes Basics and Architecture ⚙️
  - Why Use Kubernetes?
    - Kubernetes manages containerized microservices applications.
    - Containers isolate code and dependencies, enabling easy scaling and portability.
    - Containers are ephemeral and need effective management for resilience.
  - Components Overview
    - Pods: Wrappers around containers; Kubernetes manages pods, not containers directly.
    - Nodes: Virtual machines running pods; grouped into clusters.
    - Two types of nodes:Worker nodes host pods.Controller node manages cluster operations.
      - Worker nodes host pods.
      - Controller node manages cluster operations.
  - Communication and Tools
    - kubectl: CLI tool to access clusters via the controller node and API server.
    - API server validates and routes requests, communicating with nodes and ETCD.
    - ETCD: A key-value store that saves cluster state and metadata.
    - Scheduler assigns pods to appropriate nodes.
    - Controller manager oversees cluster behavior and resource control.
    - Kubelet runs on every node to manage containers using container runtimes like Docker or containerd.
- Managing Kubernetes Resources 📦
  - Manifests and YAML
    - Manifests define Kubernetes objects declaratively in YAML format.
    - Key YAML structures: lists (sequences) and dictionaries (mappings).
    - Careful indentation defines relationships and ownership.
    - Manifests must specify API version, kind, metadata, and spec.
  - Core kubectl Commands
    - kubectl get lists resources.
    - kubectl describe provides detailed info and event logs.
    - kubectl apply creates or updates resources from manifests.
    - kubectl delete removes resources by file or name.
  - Namespaces and Resource Quotas
    - Namespaces isolate resource groups.
    - Objects deploy into namespaces by default or via metadata.
    - Resource quotas limit CPU and memory per namespace to control usage.
- Pods, Containers, and Probes 🫧
  - Pod Lifecycle and Resource Control
    - Containers run inside pods; container lifecycle still applies.
    - Containers require images pulled from registries.
    - Probes monitor container health:Liveness probes: Restart unhealthy containers.Readiness probes: Control traffic flow to containers not ready.
      - Liveness probes: Restart unhealthy containers.
      - Readiness probes: Control traffic flow to containers not ready.
  - Working with Pods
    - Pods can be run imperatively or created with manifests.
    - Pod states include creating, running, and error/terminated.
    - Important to monitor container restarts as red flags.
- Config Maps, Secrets, and Volumes 🎁
  - Config Maps
    - Store configuration data external to containers.
    - Mounted as volumes to pods for shared and persistent config.
    - Supports file-based or environment variable injection.
  - Secrets
    - Like config maps but store sensitive data (passwords, keys).
    - Not encrypted by default; further encryption needed externally.
    - Used for environment variables or mounted volumes.
  - Storage Management
    - Persistent Volumes (PV) represent physical storage.
    - Persistent Volume Claims (PVC) request and bind to PVs.
    - Storage Classes define storage provider and type.
    - Volumes mounted inside pods to provide persistent data.
    - Node selectors can enforce pod location for storage proximity.
- Deployments, Replica Sets, and Scaling 🚀
  - Deployments
    - Manage stateless applications by creating Replica Sets.
    - Provide scaling and self-healing of pods.
    - Support rolling updates with zero downtime.
    - Track deployment history enabling rollbacks.
  - Replica Sets
    - Older Kubernetes resource managing pod replicas.
    - Deployments create and manage Replica Sets automatically.
  - Labeling and Selectors
    - Labels are key-value metadata tags on objects.
    - Used by deployments, services, and selectors to group and manage resources.
    - Kubectl allows label creation, modification, and filtering.
- Networking and Services 🌐
  - Pod Networking Challenges
    - Pods have ephemeral IPs that change on restart.
    - Communication requires stable endpoints.
  - Services
    - Provide consistent IP and DNS to route traffic to dynamic pods.
    - Types:ClusterIP: Exposes service internally inside cluster.NodePort: Exposes service on a static port on each node, accessible externally.LoadBalancer: External load balancer provisioned via cloud provider.
      - ClusterIP: Exposes service internally inside cluster.
      - NodePort: Exposes service on a static port on each node, accessible externally.
      - LoadBalancer: External load balancer provisioned via cloud provider.
    - kube-proxy runs on nodes, manages routing rules, and load balances traffic.
    - Services route requests with round-robin by default across pods.
  - Network Policies
    - Define ingress and egress rules for pods within namespaces.
    - Control which pods or IP blocks can communicate.
    - Default deny behavior restricts traffic unless explicitly allowed.
- Logging and Troubleshooting 🔍
  - Each container generates logs via standard output and error streams.
  - Use kubectl logs to access logs per pod or individual containers.
  - Logs are ephemeral; deleted when pods are removed.
  - Third-party tools often used for log aggregation and persistence.
  - Use kubectl describe and kubectl logs to troubleshoot pods and errors.
- Summary and Recommendations 🚀
  - Kubernetes fundamentals covered from architecture to advanced topics like deployments and networking.
  - Mastery requires hands-on labs, practice, and deeper learning.
  - Alta3 Academy offers extended courses, AI and human support, and exam preparation resources.
  - Strong emphasis on understanding YAML, manifests, and essential kubectl commands.
  - Focus on resource management, scaling, and security with network policies.

[NoteGPT] AI Mind Map Generator